

```

---
title: "Race Condition in Stock Sales: From Bug to Solution"
published: false
description: "Lab 05 demonstrates how concurrent READ-CHECK-WRITE corrupts inventory invariant, and how atomic update, row lock, and UNIQUE constraint prevent lost update."
tags: race-condition, concurrency, golang, database
---
```

Race Condition pada Penjualan Stok

Two cashiers, one sparepart in stock, both press "Pay" simultaneously. Final result: 2 successful sales, final stock 0. The invariant `initial == success + final` is broken: `1 != 2 + 0`.

This is the exact scenario Lab 05 reproduces deterministically using Go channel barriers and 500 concurrent goroutines.

The Read-Check-Write Pattern

Every business flow that touches shared mutable state follows this shape:

```

```
READ stock
CHECK stock > 0?
WRITE new stock
```
```

The gap between CHECK and WRITE is the race window. Go's `go test -race` does not catch this – all in-memory variables are thread-safe; the corruption happens at the database level.

Deterministic Reproduction

`TestLostUpdate_Deterministic` uses a channel barrier to force the interleaving A READ → B READ → A WRITE → B WRITE. The test PASS means the invariant broke – exactly as designed for the unsafe implementation.

```

```go
// UnsafeInventory: READ → CHECK → WRITE is not atomic.
func (r *UnsafeInventory) TrySell(ctx context.Context, productID string) error {
 stock, _ := r.GetStock(ctx, productID)
 if stock <= 0 {
 return ErrOutOfStock
 }
 return r.SetStock(ctx, productID, stock-1)
}
```
```

Safe Alternative 1: Atomic Conditional Update

```

```sql
UPDATE inventory_products
SET stock = stock - 1
WHERE id = $1 AND stock > 0
RETURNING stock;
```
```

1 row affected = success, 0 rows = out of stock. No SELECT before UPDATE, no stale read window. `TestAtomicUpdate_HighContention` proves invariant holds with 500 goroutines and initial stock 100 (success=100, rejected=400).

Safe Alternative 2: Row Lock

`SELECT ... FOR UPDATE` takes a row-level lock inside a transaction. Transaction B blocks until A commits. `TestPostgresRowLock\_ConcurrentStock` verifies blocking via `pg\_stat\_activity` – not inferred, directly observed.

Safe Alternative 3: UNIQUE Constraint

For booking slots, `UNIQUE(branch\_id, service\_date, slot\_time)` is the final safety net. `TestPostgres\_ConcurrentBooking`: 500 goroutines, created=1, conflict(SQLSTATE 23505)=499. The database engine rejects the duplicate – even if application logic has a bug.

What Transaction Alone Does Not Fix

On default `READ COMMITTED`:

```
```sql
BEGIN;
SELECT stock FROM products WHERE id = 1; -- reads 1
-- application checks stock > 0
UPDATE products SET stock = 0 WHERE id = 1;
COMMIT;
```

Both transactions read 1, both update to 0. Lost update. Correctness depends on query pattern, lock, and isolation semantics – not just BEGIN/COMMIT.

## ## Common Mistakes

- Frontend validation only (bypass via API).
- No UNIQUE constraint (SELECT before INSERT).
- Assuming BEGIN/COMMIT is sufficient.
- Using mutex for microservices (single instance, not all servers).

## ## Source

Implementation lives in `labs/05-race-condition` of the Software Engineering Lab repository.

<https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/05-race-condition>